

Full Stack Development with MERN

Project Documentation format

1. Introduction

Project Title: Automating Employee Data Management System

Team Members:

- Member 1 – Frontend Developer
- Member 2 – Backend Developer
- Member 3 – Database Manager
- Member 4 – Testing & Documentation

2. Project Overview

Purpose:

The purpose of this project is to develop a web-based application that automates the management of employee data. It aims to reduce manual work, improve accuracy, and provide easy access to employee information.

Features:

- User Registration and Login
- Employee Data Management (Add, Update, Delete)
- Attendance Tracking
- Dashboard with analytics
- Report Generation
- Secure Authentication

3. Architecture

Frontend (React):

The frontend is built using React.js. It provides a dynamic and responsive user interface. Components are used to manage different parts of the UI such as login, dashboard, and employee management.

Backend (Node.js & Express.js):

The backend is developed using Node.js and Express.js. It handles API

requests, business logic, authentication, and communication with the database.

Database (MongoDB):

MongoDB is used to store employee data. Collections are used to store user details, employee records, and attendance data. Mongoose is used for schema modeling and database interaction.

4. Setup Instructions

Prerequisites:

- Node.js
- MongoDB
- npm (Node Package Manager)
- Code Editor (VS Code recommended)

Installation:

1. Clone the repository:
`git clone https://github.com/your-repo/project.git`
2. Navigate to project folder:
`cd project`
3. Install dependencies:
 - For client: `cd client && npm install`
 - For server: `cd server && npm install`
4. Set environment variables (e.g., MongoDB URI, JWT secret).

5. Folder Structure

Client (Frontend):

- components/
- pages/
- services/
- App.js

Server (Backend):

- routes/
- controllers/
- models/
- config/
- server.js

6. Running the Application

- Frontend:
Run npm start in the client directory
- Backend:
Run npm start in the server directory

The application will run locally on:

- Frontend: http://localhost:3000
- Backend: http://localhost:5000

7. API Documentation

1. User Registration

- Method: POST
- Endpoint: /api/register
- Request: { name, email, password }
- Response: User created successfully

2. User Login

- Method: POST
- Endpoint: /api/login
- Request: { email, password }
- Response: JWT Token

3. Add Employee

- Method: POST
- Endpoint: /api/employees
- Request: Employee data
- Response: Employee added

4. Get Employees

- Method: GET
- Endpoint: /api/employees
- Response: List of employees

8. Authentication

Authentication is implemented using JWT (JSON Web Tokens).

- Users log in with email and password
- A token is generated and sent to the client
- Protected routes require a valid token
- Middleware is used to verify authentication

9. User Interface

The UI is designed to be simple and user-friendly. It includes:

- Login & Registration pages
- Dashboard with analytics
- Employee management interface

10. Testing

Testing is done using manual testing and basic validation checks.

- Functional testing for all features
- Input validation testing
- Performance testing for response time

10.Screenshots or Demo

- Dashboard view
- Employee management page
- Login page

12. Known Issues

- Minor UI alignment issues in some pages
- Limited scalability in current version
- Requires internet connection for full functionality

13. Future Enhancements

- Add role-based access control (Admin/User)
- Integrate cloud storage
- Add mobile application support
- Implement advanced analytics and AI features